

AttestGuard

Trade-finance advances, funded only when the underlying event is cryptographically verified - and only when a deterministic, on-chain policy says the amount is safe to release.

Version 0.2 (v2 contract revision) - August 2026

Abstract

Trade finance - a supplier advancing goods against an invoice, then waiting 30-90 days (or paying a discount to a factoring desk) to get paid - is still, in 2026, gated on someone manually checking that delivery happened before money moves. That check is either slow (a human reviewing paperwork) or centralized (an oracle operator whose word you have to trust).

AttestGuard removes the trust requirement from the first half of that problem and the manual-review requirement from the second half. It uses Creditcoin's Attestcoin Protocol to cryptographically verify that a delivery-confirmation event really happened on a source chain, with no oracle operator in the loop - and it gates every fund release behind a deterministic, on-chain policy contract that an AI agent orchestrating the process cannot bypass, only fail to use correctly. The result is an AI agent that can be trusted to move real money, not because you trust the agent, but because you don't have to: cryptography checks the input, immutable contract logic checks the output.

This document describes what is actually built and deployed - not a roadmap dressed up as a product. Every mechanism described here has a corresponding line of code and, in most cases, a live testnet transaction demonstrating it. Where something is not yet built, it is labeled as such.

Problem Statement

The event-verification problem. A financing decision that depends on "did the buyer receive the goods" needs a source of truth for that question. Today that source is almost always a person, a centralized API, or an oracle network whose incentive alignment you have to evaluate separately from the financing logic itself. Every one of those is a single point of failure or a single point of trust.

The AI-agent-authority problem. As AI agents increasingly initiate on-chain financial transactions - reading data, forming a judgment, and acting on it - the industry pattern has been to trust the agent's judgment directly, often based on a statistical "risk score" that turns out, on inspection, to be built on placeholder data (a pattern documented in this author's own earlier project, Agentic Wallet Guardian, and named explicitly as the reason for its successor, Agent Guardrail). An agent that is wrong, buggy, or compromised should not be able to move money outside pre-agreed bounds - full stop, regardless of how good its reasoning looked.

Why both at once. Solving only the first problem gets you a trustless oracle with no safety rails - a verified event still doesn't tell you whether releasing this amount, to this party, right

now, is sound financial policy. Solving only the second gets you a well-guarded agent with nothing real to act on. AttestGuard exists because both halves were already solved separately (Attestcoin Protocol for the first, this author's Agent Guardrail project for the deterministic-policy pattern) and needed to be composed.

Vision & Mission

The mission is narrow on purpose: make "an AI agent autonomously financing real-world trade" something a lender, a supplier, or an auditor can verify independently, rather than something they have to take on faith.

In three to five years, the expectation is that AI agents managing short-duration, event-triggered financial flows - invoice advances, escrow releases, insurance payouts contingent on a verifiable real-world event - become common enough that the interesting question stops being "can an agent do this" and becomes "how do we know it did this correctly, every time, without a human checking each transaction." AttestGuard's answer - cryptographic input verification plus immutable on-chain policy enforcement - is a pattern intended to generalize past trade finance to any AI-agent-triggered action gated on a real-world event: insurance claims, supply-chain milestone payments, service-level-agreement penalties.

Market Context

Trade finance is a multi-trillion-dollar market with a persistent gap: the World Trade Organization and multiple industry surveys have repeatedly cited a trade finance gap in the hundreds of billions of dollars, disproportionately affecting small and medium suppliers who can't access traditional factoring at reasonable rates because the manual verification and underwriting cost doesn't scale down to smaller invoice sizes. Automating the verification step - without removing the safety rails that traditional underwriting provides - is what makes smaller, faster, cheaper advances economically viable.

Separately, Creditcoin's own positioning is explicitly built around real-world credit and lending infrastructure, and the Attestcoin Protocol is new infrastructure (this project is built for a hackathon centered on it) specifically aimed at proving cross-chain events without a centralized oracle. AttestGuard is a concrete application of that infrastructure to a market Creditcoin is already oriented toward, rather than a generic "AI plus blockchain" demo bolted onto an unrelated chain.

Technical Architecture

1. Buyer confirms delivery on source chain
 (TradeConfirmation.sol)
 |
 v
2. Attestcoin Protocol proves it happened
 (no oracle operator)
 |
 v
3. AttestGuardManager.sol (on Creditcoin)
 |-- within caps? --> auto-fund
 |-- over cap? --> flag for human guardian

```
| -- repaid?      --> raise cap  
    (reputation, earned only from  
    verified history)
```

Components:

- TradeConfirmation.sol (source chain, Ethereum Sepolia in the current deployment) - a minimal contract with no knowledge of Creditcoin or AttestGuard. Buyers call confirmDelivery(invoiceId, supplier, amount) and it emits a plain event. This is deliberate: the contract being proven against doesn't need to be aware it's being proven against, which is the entire value proposition of the Attestcoin Protocol over a traditional oracle (no integration burden on the source-chain contract).
- AttestGuardManager.sol (Creditcoin CC3 testnet) - an Attestcoin Smart Contract (ASC). Its fundAdvanceFromQuery entry point takes a Merkle inclusion proof and a continuity proof, calls Creditcoin's native Block Prover precompile (fixed address 0x0FD2) to verify them, decodes the underlying transaction bytes via EvmV1Decoder (from the official @gluwa/usc-contracts package), and matches the resulting DeliveryConfirmed log against a registered invoice before running the guardrail policy gate described below.
- offchain-agent/ (TypeScript) - the orchestration layer. Watches the source chain for events, requests proofs from Creditcoin's Attestcoin prover service, runs an advisory (gas-saving, non-authoritative) policy pre-check, optionally asks an LLM to write a human-readable explanation of the decision for a dashboard, and submits the proof on-chain. Nothing in this layer holds funds or has unilateral authority - see Core Technology below for exactly where its authority ends.

Data flow for a single advance:

1. Owner registers an invoice on AttestGuardManager (supplier, buyer, invoice amount, requested advance amount).
2. Buyer calls confirmDelivery on the source chain when goods are received/accepted.
3. The off-chain agent detects the event, waits for Creditcoin to attest the containing block, and fetches a proof from the Attestcoin prover service.
4. The agent submits the proof to fundAdvanceFromQuery. The contract independently re-verifies it via the precompile - the agent's own proof fetch is not trusted, only re-checked.
5. The contract decodes the transaction, matches the log against the registered invoice, and runs the policy gate: within the supplier's auto-approve cap and daily cap -> immediate transfer; over either -> PendingConfirmation, requiring a guardianConfirmer transaction.
6. On repayment (currently owner-attested; see Security Model for the honest caveat on this step), the supplier's auto-approve cap grows for next time.

Core Technology

Where AI ends and enforcement begins. This is the single most important design decision in the system, and it is enforced at the compiler level, not by convention: the off-chain agent's policy check in `policy.ts` is a pure function with no side effects - it returns a verdict string, nothing more. It cannot call the contract, cannot move funds, and its result is used only to decide whether it's worth spending gas on a proof submission that the contract might reject anyway. The actual, irreversible decision happens in `_applyPolicyAndMaybeFund` inside `AttestGuardManager.sol` - a view-adjacent internal function whose inputs are entirely on-chain state (registered advance amounts, current caps, today's funded total), not anything the agent asserts at call time. An agent that is compromised, buggy, or actively malicious can at most submit a proof the contract then rejects; it cannot construct a call that skips the gate, because the gate isn't a parameter, it's the function itself.

The LLM's authority is zero. `explain.ts` is the only place an LLM is called anywhere in the system. It is given the deterministic verdict *after* it has already been decided, and asked only to phrase it in plain language for a human dashboard. If the LLM call fails, times out, is disabled, or returns garbage, the pipeline proceeds identically - the funding decision was made before the call and does not depend on its result. This is a direct, structural answer to prompt-injection and agent-hijacking concerns raised in AI-agent security research generally: there is no prompt for an attacker to inject that would change what money does, because the component that talks to an LLM has no path to money.

Reputation as an earned, not asserted, primitive. A supplier's auto-approve cap only grows via `acknowledgeRepayment`, and every growth event is a public, indexed `AutoApproveCapUpdated` log - a supplier's entire autonomy history is auditable from chain data alone, without trusting a database. The honest caveat (see Security Model) is that this specific function is currently owner-attested rather than itself proof-gated; closing that gap is the highest-value remaining architectural change, and is scoped explicitly in the roadmap rather than glossed over.

Defense in depth on the funding path. `nonReentrant` on both fund-moving functions, `Pausable` as a circuit breaker independent of the registration path (so a paused contract can still register future invoices while an incident is investigated), replay protection via a `processedQueries` mapping keyed on chain, block height, and transaction index (so the same proof cannot fund twice), and a hard `globalMaxAdvance` ceiling that no per-supplier cap can override.

Security Model

Full technical detail lives in [SECURITY.md](#) in the repository and is kept up to date there as the single source of truth; this section summarizes the trust boundaries.

What Attestcoin Protocol guarantees: the source-chain event really happened, verified via Merkle inclusion and continuity proofs against Creditcoin's own consensus-adjacent infrastructure. No off-chain party, including this project's own agent, can forge this.

What it does not guarantee: that the invoice being financed is a real business relationship, or that the party who triggered the event is who your business process expects - that's a

registration-time trust assumption (see below), not something Attestcoin Protocol is designed to solve.

What is currently centralized, stated plainly rather than hidden: invoice registration (`registerAdvance`) and repayment acknowledgment (`acknowledgeRepayment`) are owner-controlled. This is the direct, intentional scope boundary of the current version - the parts that *are* decentralized (event verification, policy enforcement) are the parts Attestcoin Protocol and immutable contract logic are actually good at; the parts that require a real-world trust decision (is this invoice legitimate, did repayment really happen) are left as an explicit, auditable admin action rather than papered over with a false decentralization claim.

Why this can't be copied in a week: not because the individual pieces are exotic - Attestcoin Protocol's SDK and the Block Prover precompile are Creditcoin's own, openly documented infrastructure - but because the composition decision (advisory-only AI, unbypassable on-chain policy, explicit and audited trust boundaries rather than marketing-copy decentralization) is a discipline, not a library. It was arrived at directly through two prior projects and an external audit that caught the overclaiming pattern this project was built to avoid repeating.

Use Cases

A small textile supplier, first-time relationship. Ships goods to a new buyer, requests a modest advance against the invoice. The system starts them at `DEFAULT_AUTO_APPROVE_CAP` with no manual review - the first several advances with any given buyer relationship are exactly the case the default cap is sized for.

A recurring supplier-buyer pair with a repayment track record. Each `acknowledgeRepayment` raises the supplier's auto-approve cap, so a supplier who reliably gets repaid needs less manual guardian intervention over time - autonomy that is earned from on-chain history, not asserted.

A lender/liquidity provider. Deposits stablecoin liquidity via `depositLiquidity` (or, in the v2 revision, can reclaim undeployed liquidity via `withdrawLiquidity`) without needing to underwrite each individual advance manually - the policy gate is the underwriting logic, applied consistently and auditable after the fact.

A guardian/risk reviewer. Only sees the advances the deterministic policy already flagged as exceeding a cap - not a queue of every transaction, which is what makes human review economically viable at volume.

Roadmap

Shipped (v2, current deployment): Attestcoin-verified funding, on-chain guardrail policy with auto-approve/daily/global caps, human-confirmation path for over-cap advances, reputation growth on repayment acknowledgment, `Pausable` circuit breaker, `withdrawLiquidity` escape hatch, full test coverage including a mock-precompile end-to-end suite (17/17 passing) exercising the real proof-decode-policy path inside CI.

Next (proof-gating the remaining centralized steps): bind `acknowledgeRepayment` to a verified proof of the buyer's on-chain repayment, the same way funding itself is proof-gated - closing the last honestly-documented centralization gap. Supplier/buyer self-registration with staking or a lightweight KYC gate, reducing reliance on owner-asserted invoice legitimacy.

Later: multi-source-chain support (the current deployment trusts one `sourceChainKey`), a hosted guardian dashboard reusing the confirmation-UI pattern from this author's Agent Guardrail project, and a buyer-relationship history index replacing the current placeholder logic in the off-chain agent.

Future Research Directions

The advisory-AI / unbypassable-policy split used here for trade finance is not specific to trade finance. The same pattern - an LLM or agent framework proposes and explains, a deterministic on-chain (or otherwise immutable) policy gate independently decides - applies to any domain where an AI agent's judgment needs to inform, but not unilaterally control, an irreversible action: insurance payout triggers, supply-chain milestone payments, automated compliance actions. Generalizing the policy-gate contract into a reusable primitive, parameterized by event type rather than hardcoded to `DeliveryConfirmed`, is the natural next research direction once the trade-finance-specific version has real usage data to learn from.

Conclusion

The honest version of this project's pitch is not "AI plus blockchain removes trust from finance" - that claim is bigger than what's actually built, and this document has tried throughout to be explicit about where trust is still required (invoice registration, repayment acknowledgment) versus where it has genuinely been removed (event verification, policy enforcement). What AttestGuard demonstrates concretely is narrower and, for that reason, more credible: that an AI agent can be given real operational authority over financial transactions specifically because the authority is bounded by cryptography on one side and immutable contract logic on the other, not because the agent is trusted to behave. That is a pattern worth building on, and this repository is the working implementation of it, not a description of one.